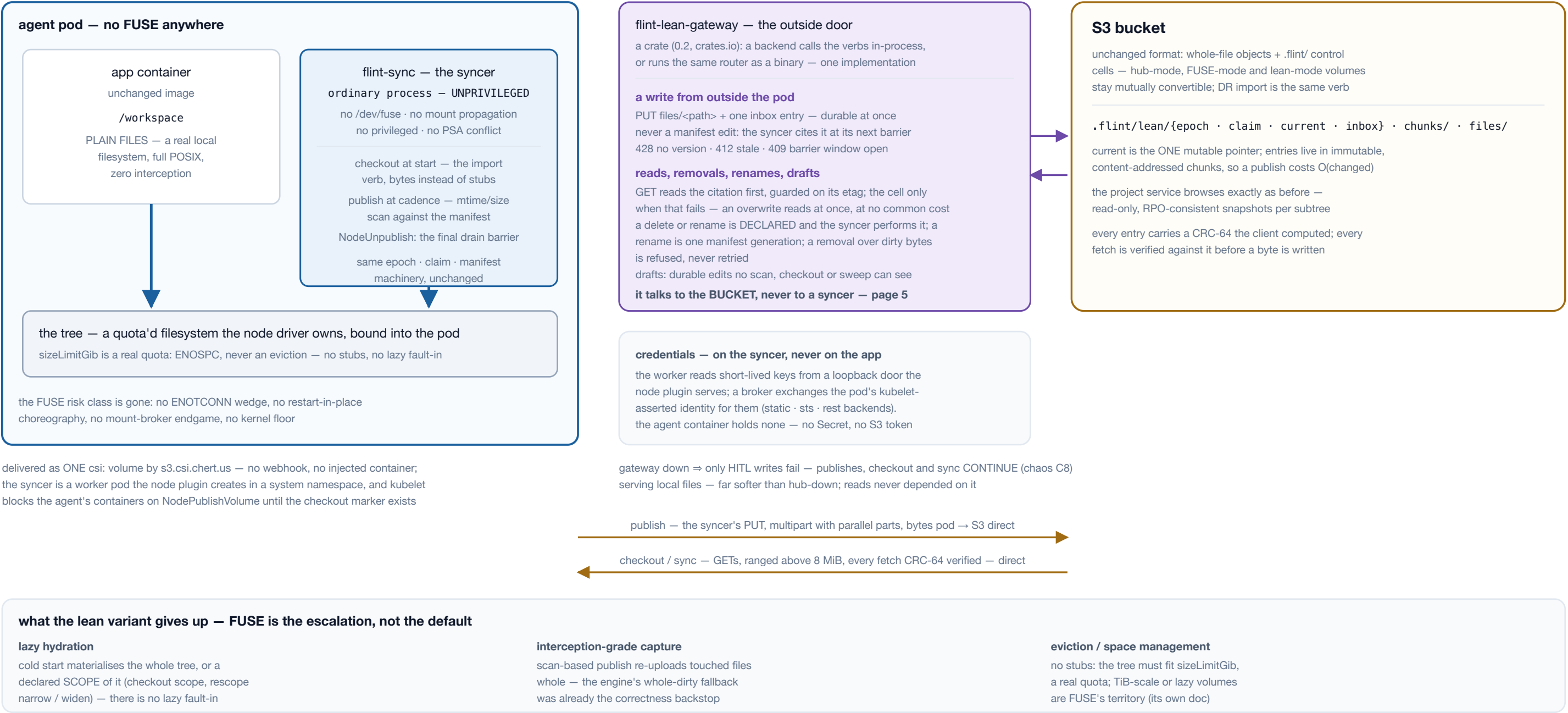


The third front end over the flint bucket format, split out of the flint-fuse architecture doc: if the workspace fits on disk, no interception is needed at all. Materialize plain files at pod start, publish changed files at a boundary with an unprivileged **syncer** — a worker pod the CSI node driver starts beside the tenant, never a container injected into it — and give writers outside the pod a stateless gateway that is, since 0.2, a library. Most of FUSE's value at a fraction of its risk surface — same bucket format, same engine.



The decision of record: every REST surface in the system speaks the S3 dialect — the hub's file API converges onto it, the gateway serves it, and direct-mode non-mount access (lean and FUSE alike) is plain S3 against the bucket. One library, three placements; the consistency contracts say which view a client is reading.



Pages 1–2 describe publishing **at cadence**: an mtime/size scan on a timer. That is still the default, and it is still the whole story for a workspace nobody is watching. What it could not express is a coherent point — “these files, together, now” — so an agent finishing a unit of work had no way to say so and no way to learn when its bytes were durable. The boundary verbs are that sentence, written as files in the workspace the agent already has. Nothing new is mounted, no port is opened, and a workspace that ignores them pays nothing: measured at **20 bucket requests per 22 s idle with the verbs off and 20 with them on**.



Lean is not a general filesystem and does not try to be. It is for **one agent, one workspace, one writer, a workspace that fits on disk** — the harness shape: clone-ish tree of code and artifacts, an agent that reads and writes it locally at full speed, and a durable point somebody else can read when the work is done. Everything below is the boundary of that claim, in measured numbers, so the answer to “will this work for us” is arithmetic rather than opinion.

three questions, in order

1 · does the workspace fit the disk?

the whole tree is materialized — no stubs, no eviction.  
No ⇒ FUSE (its own doc). This is the hard gate.

2 · is merge-at-the-manifest enough?

writers share a subtree and merge per barrier; a same-file edit is last-boundary-wins. No ⇒ the hub, for live shared POSIX.

3 · is snapshot freshness acceptable?

readers see the last published boundary, not your last write. No ⇒ the hub. Yes ⇒ lean, and read on.

the loop — one agent, start to durable

pod starts

the node driver starts the syncer;  
checkout runs BEFORE the  
agent's first line

agent works

plain local files, full POSIX,  
zero interception — this is  
the part that is just fast

agent declares

echo > .flint/publish  
one file write — the whole  
API surface

syncer answers

.flint/publish.ack  
ok ⇒ the bytes are in S3 and  
a sibling can read them

what it costs — MEASURED 2026-08, loopback floors (0b), before the read-path work on page 6; a latency-bound proxy moves these

|                                                                                                          |                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| bytes                                                                                                    | checkout 3.3 s/GiB · publish 8.0 s/GiB                                                                                                        |
| 100k files                                                                                               | checkout 49.5 s · first publish 65 s (was 117 s before bounded fan-out) · idle tick 1.85 s                                                    |
| 1M files                                                                                                 | checkout 7 m 05 s · a 264 MiB manifest then — chunked since 1.45.0: the pointer is a few hundred bytes, a publish costs O(changed)            |
| the cap                                                                                                  | the disk (sizeLimitGib, a real quota) and the time to materialise the file count — not the manifest; one chunk level is sized to ~10M entries |
| idle costs ~5 bucket requests per tick, and the verbs add ZERO when unused — measured 20 vs 20 over 22 s |                                                                                                                                               |

worked example — a coding-agent fleet

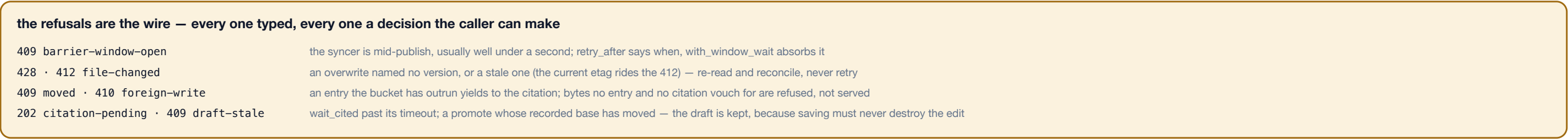
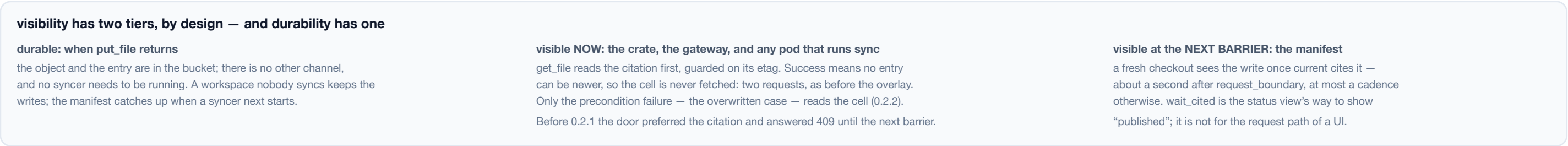
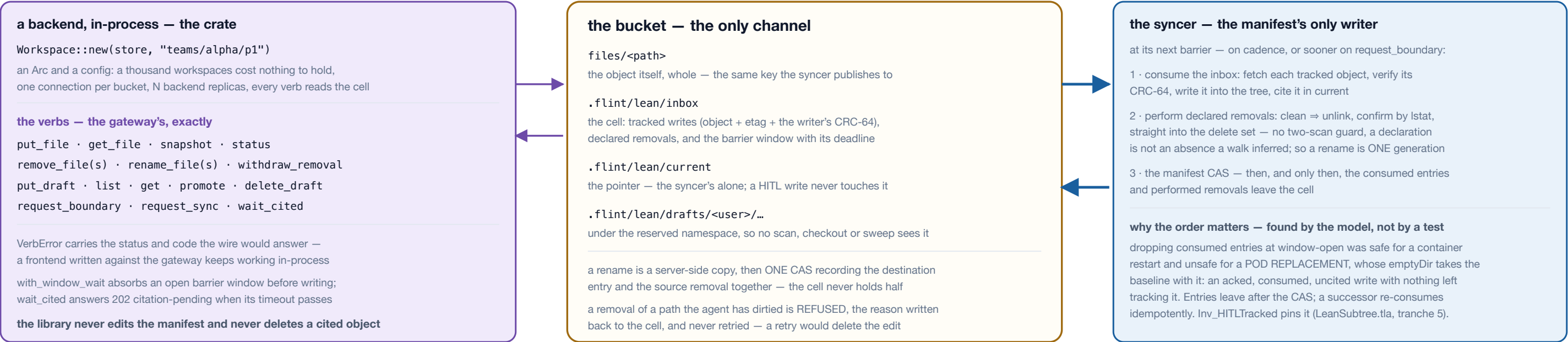
A 2 GiB repo, 40k files, 30 agent pods, each on its own prefix. Checkout ≈ 7 s of bytes plus ~20 s of file count, so a pod is working inside half a minute. Each agent publishes when it finishes a task rather than on a timer, so a reviewer reads whole units of work, never a half-written tree. Idle pods cost about five requests a tick each. Nothing is shared live: agents on one workspace merge at each barrier, disjoint files cleanly and a same-file edit as last-boundary-wins with the loser preserved — live shared POSIX is the hub's job, not this one.

when to walk away from lean

- The tree does not fit the pod's disk, or is TiB-scale, or you need lazy access to a small slice of a huge dataset — FUSE.
- Two writers must see each other's bytes live, or you need byte-range locks, O\_EXCL or shared sqlite/git ACROSS pods — the hub.
- A reader cannot tolerate seeing the last boundary instead of the last write — the hub.

**The fit question is one question, and it is about the disk.** Everything else follows: because the whole tree is local, the app gets real POSIX at local speed and the FUSE risk class disappears; because it is local, it must fit; and because publishing is a boundary rather than a write-through, a reader sees coherent points instead of a live tree. **The file count binds before the byte count, but no longer through the manifest.** In 2026-08 the manifest was one object rewritten whole — 264 MiB at a million entries, so the cap was expressed in files (~250k). Since 1.45.0 the entries live in immutable, content-addressed chunks named by a pointer of a few hundred bytes: a publish sends only the chunks whose address the pointer does not already hold, a small project is one chunk and pays what the single generation did, and the design sizes one chunk level to about ten million entries. What still binds is the disk, now a real quota, and the time to materialise and scan the count — which page 6 measures. **The verbs change the shape of the workflow, not its cost.** Cadence alone forces a choice between publishing too often (wasted work) and too rarely (a reviewer reads a half-finished tree); a declared boundary removes the choice, and costs nothing when unused. **Say no early.** The three questions above are ordered so the cheapest disqualifier comes first, and a workload that fails any of them is better served by another front end over the same bucket format — all three are mutually convertible, so choosing lean today does not strand the data if the answer changes.

A UI, a reviewer, a backend serving a thousand workspaces: none of them is in the pod, and none of them may touch the agent's tree or edit the manifest. **flint-lean-gateway is the door for them, and since 0.2 it is a library** — a backend depends on the crate and calls the verbs in-process; the flint-lean-gateway binary is main around the same router, so there is one implementation and one set of refusals. The door talks to the **bucket**, never to a syncer: it can run on another cluster, or on none.



**What a write is.** put\_file returns when the object is at <prefix>/files/<path> and an entry naming it is in the inbox cell. Both are in the bucket; that is the whole durability promise, and it is immediate. What the library will *not* do is edit the manifest: only a syncer holding the publish fence for its commit section writes the manifest, and that is what the protocol's model checks — a manifest writer outside the fence would reintroduce exactly the race the barrier exists to prevent. **A delete from outside is declared, never performed.** A cited object deleted by a caller wedges every checkout with “the manifest cites it but it is gone”, so remove\_file records the intent and returns; the listing hides the path at once, the bytes stay readable by name until the syncer performs it, and a removal the syncer refuses leaves the file exactly as it was — there is no function in the crate that deletes a cited object. **A rename is one generation.** The server-side copy lands first (a 10 GB checkpoint moves without a download), then one CAS records the destination entry and the source removal together, and the barrier cites both halves in one manifest generation: a reader sees the old name or the new, never both and never neither. **Drafts are the edit that must not be published yet** — under the reserved namespace, where no scan, checkout or sweep can see them; the base etag is recorded, never enforced at save time, and a promote is a HITL write conditioned on it. **The cost of the overlay on the common read is zero**, and it is pinned by a counting store: (inbox fetches, object fetches) of (0, 1) for an unmodified cited file, (1, 2) for an overwritten one; the 0.2.1 order fails the first count. Crate: Lean/gateway; design of record for removals: docs/plans/flint-lean-delete-rename-design.md.

Pages 1–4 describe the protocol. This page is what changed in the syncer once it met real buckets, real nodes and the formal model: **every fetched byte is verified against a CRC the client computed**, a checkout can be a **scope** rather than the whole tree, the read and publish paths lost the copies, the lock and the double read that bounded them, and a reader **no longer stands in the writer’s queue**. Every number is a range over reps from a drill committed under `lean/e2e/perf/results/`; the four-door comparison against passthrough and the CLI is measured separately, in the door-drill results there.

integrity — the client vouches

every manifest entry carries a CRC-64, mandatory, computed by the client that moved the bytes — never copied from a backend header: Ozone returns none, and a HITL uploader may not have sent one

a fresh fetch is verified BEFORE the write: the whole-object arm hashes the body, the ranged arm folds each range in offset order; a mismatch refuses the checkout with nothing written. Consume and sync verify too

the S3 SDK validated a checksum header the store sent; a store that sends none validated nothing — exactly where the backend attests nothing, the check was empty

three citation repairs took the CRC from a HEAD; one had NO test — citing a wrong CRC there passed the battery. Six mutation controls now, each failing one test

probe-conditional · probe-copy

the verbs a deployment runs against a new backend first: a store that ignores If-Match turns every manifest CAS into last-writer-wins, silently. Ozone 2.2.1: probed

scope — part of the tree, safely

FLINT\_SYNC\_CHECKOUT\_SCOPE=inputs,docs/spec.md

checkout materialises only the citations it admits: 3 files of 2,001 on the drill, 22x faster than the tree

the admission filter is the WHOLE safety argument: a path never materialised is never cited in the baseline, so classify cannot read it as absent and the barrier cannot DELETE its object — no filter in the barrier, the sync or the delete sites; four copies would drift

flint-sync rescope inputs docs/spec.md  
flint-sync rescope --all

narrows or widens a live held set. A narrow is an UNWATCH, never an absence: the drop set is recorded in a replayable intent, not re-derived; a path with unpublished changes is refused at the door, kept on replay

two constraints: the scope matches path COMPONENTS, not prefixes; and the manifest baseline stays WHOLE — narrowing it would queue every unadmitted citation into the inbox and land it one barrier later

throughput — what was in the way

one thread, then one lock

every fetch future was polled by ONE task; the fan-out is now driven by fetchDrivers tasks (cores, ≤ 8) — which made it WORSE until mimalloc replaced musl’s one-lock malloc: 3,277–3,481 → 6,188–6,521 files/s, i4i.xlarge

the copy, and the inline write

every fetched body was copied once more, and the ranged arm wrote inline on the task that drained the fan-out: 477–500 → 654–676 MiB/s on real S3, at LESS CPU per byte

the publish read every file twice

once for its CRC, once part by part; a large object was one stream both ways. Parallel parts: publish 1.55–1.63x

defaults: fan-out 128, ranged GET above 8 MiB

small files are bound by objects in flight, not bytes; large objects fetch as parallel ranges and cap at 8 in flight

and one door left shut: FLINT\_SYNC\_RAW\_READS

GET/HEAD without the SDK, off by default: +42–56% at fan-out 512 on real S3, equal at 128 — latency-bound, it buys nothing

arbitration — a reader takes no lease

the epoch cell decides who may INSTALL a manifest. A checkout installs nothing, and was standing in that queue anyway — a deadlock wearing mutual exclusion’s clothes: a checkout waited for a publisher, a publisher for the checkout, and on a quiet cell the wait ended with a READER DEPOSING A LIVE PUBLISHER, sixty seconds in

mid-read safety never was the lease’s job: every fetch carries the cited etag as If-Match, and a publish landing mid-checkout now names the publisher and says to re-run

four syncers reading disjoint subtrees of one prefix: 2.1x one syncer with the claim gone, 0.41x — slower than one — with it

barrier · sync · rescope

still claim: they write, or rewrite the held set and leave a replayable intent behind

the model is the gate — lean/formal, TLA+ checked before the code, and the code written to it

check.sh runs 94 configurations, ALL required: 20 strict runs that must hold, and mutations and probes that must FAIL — a model that cannot rediscover its own bug classes proves nothing, and a probe names an action through a ghost only that action writes. LeanSubtree.tla is the protocol (lease, barrier, inbox, removals, renames); LeanChunkGC.tla and LeanChunkMerge.tla are the chunked manifest’s reaper and its 412 merge. Three times the abstraction found what no test had:

the early inbox drop

consumed entries left the cell at window-open, “durably in the baseline” — true of a restart, false of a pod replacement. A hand-simulated crash test PASSED with the bug; the real barrier driven into an injected CAS failure did not. Entries now leave after the CAS

the reaper’s ordering rule

“list candidates, then union the references” is not what makes GC safe: the reference set must not predate a CAS the delete follows. Refs are read at the delete behind an etag fence, adoption rewrites what it adopts, and the grace outlives the longest publish

the model’s own two-scan rule

was inverted — “present at the previous scan” is a one-scan rule — and gave consumed paths no two-scan protection at all. Both corrected; a rename needs a FREE destination, or the probe is green because the action never fires. The abstraction is the bug, often

**Integrity does not depend on the client that reads, or on the store.** S3 returns a checksum header the SDK validates; Ozone carries none, so the SDK validated nothing exactly where the backend attests nothing. Now the manifest’s CRC-64 is what every reader checks — checkout, consume and sync — and it is the CRC of what the writer wrote, never a copy of what a HEAD said. Manifests written before the change do not load; lean is not yet adopted, so there is no migration. **Scope is safe because of one filter, not four.** The admission filter in `checkout` is the whole argument: what was never materialised was never cited, so nothing downstream can mistake it for a deletion — and a narrow that recorded its drop set as an intent can be replayed after a crash without re-deriving it from a tree that cannot tell a leftover from a file the agent created. **The throughput work removed work rather than redistributing it** — the copy, the inline write, the second read, the one task and the one lock — which is why the shipped defaults now equal hand-tuned settings, where before the difference was nearly 2x; the raw HTTP read path stays behind a flag because on a latency-bound node it buys nothing and the SDK’s retry and credential machinery had to be reproduced by hand, including the per-request IMDS credential defect the cluster found and a single-flight cache fixed. **The lease fences installs, not reads.** Sources: CHANGELOG 1.45.0–1.50.0 and Unreleased; docs/plans/flint-lean-scoped-read-design.md, flint-lean-chunked-manifest-design.md; lean/formal/README.md.

The obvious question once several agents share a repo: how do they integrate? The answer is that they do not integrate through flint-lean at all. **Use plain git against a real git host, and let flint-lean keep the workspace durable.** Two mechanisms, two network paths, two different jobs — and they never touch, which is exactly why nothing special is needed to make them coexist. Full guide: docs/flint-lean-git-workflow.md.



two tiers of durability — reason about a hard pod kill

- PUSHED COMMITS are safe unconditionally. They live on the git host; the pod is irrelevant to them.
  - EVERYTHING ELSE is safe to the last barrier — uncommitted edits, unpushed commits, untracked files, build state.
- A hard kill loses at most floorSecs of it (default 60 s). That is the RPO contract, and it is the reason to publish at breakpoints.
- A GRACEFUL stop loses nothing: preStop runs a final barrier. A container restart over a live tree resumes from its baseline — no re-checkout.

why NOT a git-over-S3 remote

- A lean pod reaches the bucket with the syncer's short-lived keys from the broker, scoped to its prefix — never the open bucket such a helper expects; its writes would be unscoped and carry no epoch stamp — invisible to the arbitration everything else here depends on.
- flint-lean IS a checkout/publish-to-S3 engine. A second S3 sync on top is redundant, and gives you two coherence models that disagree.
- flint-lean is ONE WRITER per subtree. N agents merging into one tree is precisely what a git remote is for — so let it be.

**The division is clean because the blind spots are complementary, not overlapping.** Git owns what has been committed and pushed — history, review, merge, and the actual integration of several agents' work. flint-lean owns everything git cannot see: the uncommitted edit, the unpushed commit, the untracked scratch file, the build cache, the sqlite database sitting next to the repo. Each agent gets its own lean workspace on its own prefix, and the agents meet at the git host. **The practical rule is a two-tier one.** Commit and push at natural breakpoints, because that is the only tier with no RPO at all; rely on flint-lean to carry the in-flight remainder across pod churn, bounded by floorSecs — or declare a boundary explicitly with `.flint/publish` when a unit of work finishes, which is page 3's subject and turns “at most 60 s of loss” into “none, and I know it landed.” **Nothing here is special-cased.** There is no git integration in flint-lean, no hook, no filter, no lock coordination — the working tree is ordinary local disk, which is the entire reason this composes.